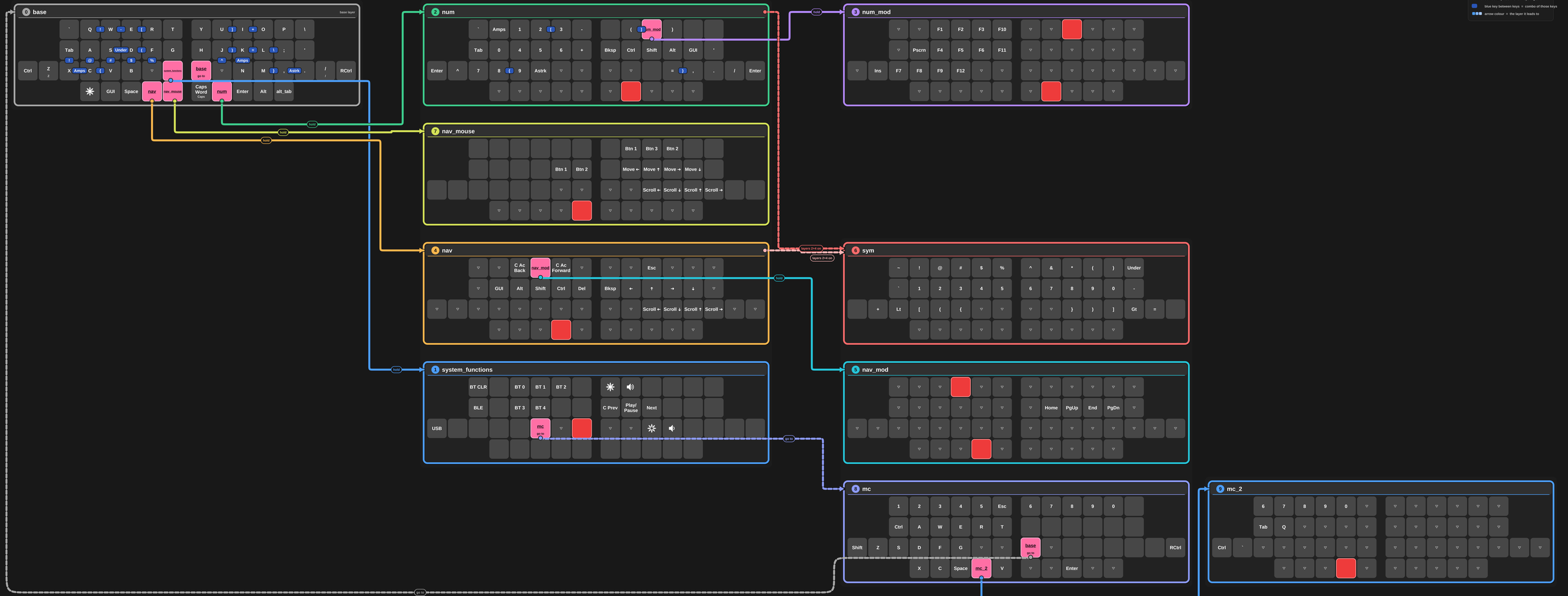
kyria_rev3.keymap

10 layers · 50 keys · 21 combos

every arrow starts on the key that triggers it

how to read

- pink key = press it to go to another layer
- violet key = press it to toggle a layer on and off
- red key(s) = hold down while you are on this layer
- solid arrow = hold
- dashed arrow = toggle / go to / one-shot
- combo = press the red keys together
- blue key between keys = combo of those keys
- arrow colour = the layer it leads to



kyria_rev3.keymap

keymap source, comments removed (bindings re-aligned as keyboard rows)

```
#include <behaviors.dtsi>
#include <dt-bindings/zmk/bt.h>
#include <dt-bindings/zmk/keys.h>
#include <dt-bindings/zmk/outputs.h>
#include <dt-bindings/zmk/pointing.h>

#define ZMK_POINTING_DEFAULT_MOVE_VAL 2000
#define ZMK_POINTING_DEFAULT_SCRL_VAL 20
#define COMBO_TIMEOUT 25

&mmv {
    time-to-max-speed-ms = <100>;
    trigger-period-ms = <5>;
    acceleration-exponent = <1>;
};

&msc {
    time-to-max-speed-ms = <150>;
    acceleration-exponent = <0>;
};

/ {
    combos {
        compatible = "zmk,combos";

        lbkt {
            bindings = <&kp LBP>;
            key-positions = <31 32>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0 2>;
        };

        lpar {
            bindings = <&kp LPAR>;
            key-positions = <18 19>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        lbrs {
            bindings = <&kp LBRC>;
            key-positions = <28 27>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0 2>;
        };

        rbkt {
            bindings = <&kp RBRT>;
            key-positions = <7 8>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0 2>;
        };

        rpar {
            bindings = <&kp RPAR>;
            key-positions = <20 19>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        rbrc {
            bindings = <&kp RBRC>;
            key-positions = <35 36>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0 2>;
        };

        dsh {
            bindings = <&mo_dsh>;
            key-positions = <2 2>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        undsc {
            bindings = <&kp UNDER>;
            key-positions = <14 15>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        amps {
            bindings = <&kp AMPS>;
            key-positions = <26 27>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        escl {
            bindings = <&kp RS(N1)>;
            key-positions = <1 2>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        plus {
            bindings = <&mo_plus>;
            key-positions = <8 9>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        eq {
            bindings = <&mo_eq>;
            key-positions = <21 20>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        ast {
            bindings = <&mo_ast>;
            key-positions = <37 38>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        bslh {
            bindings = <&kp BSLH>;
            key-positions = <23 24>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        escl2 {
            bindings = <&kp RS(N1)>;
            key-positions = <13 25>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        ats {
            bindings = <&kp RS(N2)>;
            key-positions = <14 26>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        hsh {
            bindings = <&kp RS(N3)>;
            key-positions = <15 27>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        dlr {
            bindings = <&kp RS(N4)>;
            key-positions = <16 28>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        pct {
            bindings = <&kp RS(N5)>;
            key-positions = <17 29>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        crt {
            bindings = <&kp CARET>;
            key-positions = <18 34>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };

        ampt {
            bindings = <&kp AMPS>;
            key-positions = <35 19>;
            timeout-ms = <COMBO_TIMEOUT>;
            layers = <0>;
        };
    };

    macros {
        alt_tab: alt_tab {
            compatible = "zmk,behavior-macro";
            #binding-cells = <0>;
            bindings =
                <&macro_press>,
                <&kp LALT>,
                <&macro_tap>,
                <&kp TAB>,
                <&macro_release>,
                <&kp LALT>;
            label = "ALT_TAB";
        };

        wrap_kpc: wrap_kpc {
            compatible = "zmk,behavior-macro-one-param";
            #binding-cells = <1>;
            bindings =
                <&macro_tap>,
                <&kp SPACE &macro_param_1to1 &kp MACRO_PLACEHOLDER &kp SPACE>;

            label = "WRAP_SPC";
            wait-ms = <0>;
            tap-ms = <5>;
        };
    };

    behaviors {
        th_r: th_r {
            compatible = "zmk,behavior-hold-tap";
            label = "TH_R";
            bindings = <&kp>, <&kp>;

            #binding-cells = <2>;
            tapping-term-ms = <250>;
            flavor = "hold-preferred";
            hold-trigger-key-positions = <1 25 13 26 14 2 3 4 5 15 16 17 27 28 29 12 0 24 48 42 47 41 45>;
            quick-tap-ms = <150>;
        };

        th_t: th_t {
            compatible = "zmk,behavior-hold-tap";
            label = "TH_T";
            bindings = <&kp>, <&kp>;

            #binding-cells = <2>;
            tapping-term-ms = <250>;
            flavor = "hold-preferred";
            quick-tap-ms = <150>;
            hold-trigger-key-positions = <6 7 8 9 10 18 19 20 21 22 34 35 36 37 23 11 39 38 41 42 40 45>;
        };

        brightness: brightness {
            compatible = "zmk,behavior-mod-morph";
            label = "BRIGHTNESS";
            bindings = <&kp C_BRIGHTNESS_INC>, <&kp C_BRIGHTNESS_DEC>;

            #binding-cells = <0>;
            mods = <(MOD_LSFT|MOD_RSFT)>;
        };

        c_lsft: c_lsft {
            compatible = "zmk,behavior-mod-morph";
            label = "C_LSFT";
            bindings = <&th_l LSHFT 2>, <&kp 2>;

            #binding-cells = <0>;
            keep-mods = <(MOD_RSFT)>;
            mods = <(MOD_RSFT)>;
        };

        c_rsft: c_rsft {
            compatible = "zmk,behavior-mod-morph";
            label = "C_RSFT";
            bindings = <&th_r RSHFT SLASH>, <&kp SLASH>;

            #binding-cells = <0>;
            mods = <(MOD_LSFT)>;
            keep-mods = <(MOD_LSFT)>;
        };

        mm_caps: mm_caps {
            compatible = "zmk,behavior-mod-morph";
            label = "MM_CAPS";
            bindings = <&caps_word>, <&kp CAPS>;

            #binding-cells = <0>;
            mods = <(MOD_LSFT|MOD_RSFT)>;
        };

        ws_eq: ws_eq {
            compatible = "zmk,behavior-mod-morph";
            label = "WS_EQ";
            bindings = <&kp EQUAL>, <&wrap_spc EQUAL>;

            #binding-cells = <0>;
            mods = <(MOD_LSFT|MOD_RSFT)>;
        };

        ws_plus: ws_plus {
            compatible = "zmk,behavior-mod-morph";
            label = "WS_PLUS";
            bindings = <&kp PLUS>, <&wrap_spc PLUS>;

            #binding-cells = <0>;
            mods = <(MOD_LSFT|MOD_RSFT)>;
        };

        ws_dsh: ws_dsh {
            compatible = "zmk,behavior-mod-morph";
            label = "WS_DSH";
            bindings = <&kp MINUS>, <&wrap_spc MINUS>;

            #binding-cells = <0>;
            mods = <(MOD_LSFT|MOD_RSFT)>;
        };

        ws_ast: ws_ast {
            compatible = "zmk,behavior-mod-morph";
            label = "WS_AST";
            bindings = <&kp ASTRK>, <&wrap_spc ASTRK>;

            #binding-cells = <0>;
            mods = <(MOD_LSFT|MOD_RSFT)>;
        };
    };

    keymap {
        compatible = "zmk,keymap";

        base {
            bindings = <
&kp GRAVE  &kp Q     &kp W     &kp E     &kp R     &kp T     &kp Y     &kp U     &kp I     &kp O     &kp P     &kp BSLH
&kp TAB    &kp A     &kp S     &kp D     &kp F     &kp G     &kp H     &kp J     &kp K     &kp L     &kp SEMI  &kp SQT
&kp LCTRL  &c_lsft   &kp X     &kp C     &kp V     &kp B     &trans    &mm_l     &kp N     &kp M     &kp COMMA &kp DOT   &c_rsft   &kp RCTRL
                                &brightness  &kp LEFT_GUI  &kp SPACE  &mo 4     &mo 7     &mm_caps  &mo 2     &kp RET   &kp CALT  &alt_tab
            >;
        };

        system_functions {
            bindings = <
                        &bt BT_CLR      &none     &bt BT_SEL 0  &bt BT_SEL 1  &bt BT_SEL 2  &none         &kp C_BRIGHTNESS_INC  &kp C_VOL_UP      &none          &none          &none     &none
                        &out OUT_BLE    &none     &none         &none         &none         &none         &kp C_PREV            &kp C_PLAY_PAUSE  &kp C_NEXT     &none          &none     &none
&out OUT_USB  &none     &none     &none         &none         &rt 8         &trans        &trans        &trans                &trans            &kp C_BRIGHTNESS_DEC  &kp C_VOL_DN  &none     &none     &none     &none
                                  &none         &none         &none         &none         &none         &none                 &none             &none          &none          &none     &none
            >;
        };

        num {
            bindings = <
&kp GRAVE  &kp AMPS  &kp N1  &kp N2  &kp N3  &kp MINUS     &none         &kp LPAR  &mo 3     &kp RPAR  &none     &none
&kp TAB    &kp NE    &kp N4  &kp N5  &kp N6  &kp PLUS      &kp BACKSPACE  &kp LCTRL  &kp LSHFT  &kp LALT  &kp LGUI  &kp DQT
&kp RET  &kp CARET   &kp N7  &kp N8  &kp N9  &kp ASTRK     &trans         &trans     &trans     &kp EQUAL  &kp COMMA  &kp DOT   &kp FSLH  &kp RET
                             &trans  &trans  &trans  &trans  &trans        &trans                &trans     &trans     &trans     &trans
            >;
        };

        num_mod {
            bindings = <
                        &trans    &trans    &kp F1    &kp F2    &kp F3    &kp F10       &trans    &trans    &trans    &trans    &trans    &trans
                        &trans    &kp PSCRN &kp F4    &kp F5    &kp F6    &kp F11       &trans    &trans    &trans    &trans    &trans
&trans    &kp INS   &kp F7    &kp F8    &kp F9    &kp F12   &trans    &trans        &trans    &trans    &trans    &trans    &trans    &trans
                              &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans
            >;
        };

        nav {
            bindings = <
                        &trans    &trans    &kp C_AC_BACK  &mo 5     &kp C_AC_FORWARD  &trans        &trans        &trans    &kp ESC   &trans    &trans    &trans
                        &trans    &kp LGUI  &kp LALT       &kp LSHFT  &kp LCTRL        &kp DEL       &kp BACKSPACE  &kp LEFT  &kp UP    &kp RIGHT  &kp DOWN  &trans
&trans    &trans    &trans    &trans    &trans    &trans    &trans    &trans        &trans        &trans    &mmv SCRL_LEFT  &mmv SCRL_DOWN  &mmv SCRL_UP  &mmv SCRL_RIGHT  &trans    &trans
                              &trans    &trans    &trans    &trans    &trans        &trans        &trans    &trans    &trans    &trans
            >;
        };

        nav_mod {
            bindings = <
                        &trans    &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans    &trans
                        &trans    &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans    &trans
&trans    &trans    &trans    &trans    &trans    &trans    &trans    &trans        &trans    &kp HOME  &kp PG_UP  &kp END   &kp PG_DN  &trans
                              &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans
            >;
        };

        sym {
            bindings = <
                        &kp RS(CARET)  &kp RS(N1)  &kp RS(N2)  &kp RS(N3)  &kp RS(N4)  &kp RS(N5)  &kp LS(N6)  &kp LS(N7)  &kp LS(N8)  &kp LS(N9)  &kp LS(N0)  &kp UNDER
                        &kp GRAVE  &kp N1    &kp N2    &kp N3    &kp N4    &kp N5        &kp N6    &kp N7    &kp N8    &kp N9    &kp N0    &kp MINUS
&none     &kp PLUS  &kp LT    &kp LBKT  &kp LPAR  &kp LBRC  &kp RBRC  &trans        &trans    &trans    &trans    &trans    &trans    &kp EQUAL  &none
                              &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans
            >;
        };

        nav_mouse {
            bindings = <
                        &none     &none     &none     &none     &none     &none         &none     &mkp M1   &mkp MID  &mkp M2   &none     &none
                        &none     &none     &none     &none     &mkp M1   &mkp M2       &none     &mmv MOVE_UP  &mmv MOVE_RIGHT  &mmv MOVE_DOWN  &none
&none     &none     &none     &none     &none     &none     &none     &none         &none     &mmv SCRL_LEFT  &mmv SCRL_DOWN  &mmv SCRL_UP  &mmv SCRL_RIGHT  &none     &none
                              &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans
            >;
        };

        mc {
            bindings = <
                        &kp N1    &kp N2    &kp N3    &kp N4    &kp N5    &kp ESC       &kp N6    &kp N7    &kp N8    &kp N9    &kp N0    &none
                        &kp LCTRL  &kp A     &kp W     &kp E     &kp R     &kp T         &trans    &none     &none     &none     &none     &none
&kp LSHFT  &kp Z     &kp S     &kp D     &kp F     &kp G     &trans        &trans    &trans    &none     &none     &none     &none     &kp MCTRL
                              &kp X     &kp C     &kp SPACE  &mo 9     &kp V         &trans     &trans    &kp RET   &trans     &trans
            >;
        };

        mc_2 {
            bindings = <
                        &kp N6    &kp N7    &kp N8    &kp N9    &kp N0    &trans        &trans    &trans    &trans    &trans    &trans    &trans
                        &kp TAB   &kp Q     &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans    &trans
&kp LCTRL  &kp GRAVE  &trans    &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans    &trans
                              &trans    &trans    &trans    &trans    &trans        &trans    &trans    &trans    &trans    &trans
            >;
        };
    };

    conditional_layers {
        compatible = "zmk,conditional-layers";

        sym {
            if-layers = <2 4>;
            then-layer = <6>;
        };
    };
};
```